1. Project Overview

1.1 Problem and Motivation

PhotoPilot is a proposed AI photography workflow agent that connects shooting preparation with reflection on previous sessions. Photographers often manage equipment records, technical calculators, planning notes, and shooting statistics separately. They must manually coordinate equipment choices, shooting requirements, and practical constraints while remembering lessons from earlier sessions.

PhotoPilot addresses this fragmentation by combining planning, equipment selection, technical calculations, and metadata-based review. An AI agent is appropriate because requests involve natural-language goals and competing preferences, such as reducing weight while preserving focal-length coverage. The agent will interpret these priorities and coordinate relevant resources, while deterministic software performs precise calculations and technical validation.

1.2 Target Users

The primary users are amateur and enthusiast digital photographers who own interchangeable-lens cameras and face multiple equipment choices. They need support selecting suitable equipment, preparing sessions, and applying previous experience to future plans.

Secondary users are photography students and developing hobbyists who benefit from explanations connecting photographic goals to technical decisions. The project focuses on individual photography workflows; professional studio administration, client management, image editing, and social publishing are outside its scope.

1.3 Agent Description

Users will provide a session brief describing their subjects, intended images, available equipment, and constraints. PhotoPilot will interpret the requirements, identify missing information, retrieve relevant equipment records and session history, and invoke deterministic tools. These tools will support compatibility checks and calculations involving depth of field, field of view, and exposure.

The agent will coordinate multi-step workflows by selecting appropriate tools, evaluating their results, and revising recommendations when constraints or retrieved information change. Users can revise the plan conversationally, prompting reconsideration of affected decisions.

After shooting, users may import supported photographs or metadata files. Deterministic components will extract available EXIF metadata and calculate objective statistics, including focal-length usage, camera/lens usage, and exposure-setting distributions. The agent will interpret these statistics in relation to session goals and workflow, identifying patterns useful for future equipment selection and preparation. This technical and workflow review will recognize that EXIF metadata alone cannot establish image quality, artistic success, composition quality, or whether a particular setting produced a successful photograph. Saved plans and reviews may be reused as application-level memory in future planning. A GUI and CLI will provide access to major application and agent functionality through shared services.

1.4 AI/LLM Model(s)

The initial implementation is planned to use the Google Gemini API with a Gemini-family model, with the exact model configurable through the LLM Client. The LLM will interpret natural language, reason over photography requirements and constraints, select application tools, compare alternatives, generate contextual recommendations, and revise plans using user instructions and retrieved context.

Photography calculations, equipment compatibility validation, EXIF extraction, statistical analysis, persistence, and application state management will remain deterministic. Gemini will receive and interpret structured information and deterministic tool results rather than replace precise, verifiable software operations.

If the LLM API is unavailable or fails, the system will report the error without generating unverified AI recommendations. Deterministic features, including equipment management, technical calculations, and EXIF analysis, will remain functional independently of the LLM.

1.5 Overall Architecture

PhotoPilot will use a layered, object-oriented architecture suitable for Java implementation. The presentation layer will provide the GUI and CLI. Both GUI and CLI will use the same application services and domain logic to ensure consistent functionality and validation. The application/controller layer will validate requests and coordinate operations, routing ordinary tasks to deterministic services and planning requests to the agent layer.

The agent layer will coordinate the LLM Client, history retrieval, and tool use. Domain objects and deterministic services will represent photography sessions and equipment, perform calculations, and process metadata. The data/integration layer will support persistence and access to the LLM API and an EXIF-processing library.

This separation supports later UML modeling, feature traceability, and distinct testing of deterministic software and agent behavior.

2. Detailed Feature Specifications

F01 Photography Session and History Management

Description: A session is the persistent record for a shoot. It groups its details and timeline so later features can attach shoot plans, gear selections, EXIF analysis, and reviews to the same session.

GUI Interaction: The photographer creates a session from the dashboard, saves it, and reopens it through a searchable history view.

Input: Session title, date, location (optional; user permission is required if current or precise device location is used), and optional notes or goals.

Output: A saved session with a unique ID, status, timestamps, and links to its related records.

AI Involvement: Deterministic.

Expected Workflow: Validate details → create the session → save it → show it in history → select it later to restore its details and linked work.

Error/Alternative Cases: Flag missing required details; report save or load failures without losing unsaved entries; show an empty state when history has no sessions.

F02 AI Photography Inspiration and Activity Recommendation

Description: Helps a user who wants to take photos but has no subject in mind. F02 answers "What should I shoot?"; a later shoot-planning feature answers "How should I shoot it?"

GUI Interaction: The user opens Find Something to Shoot, reviews or adjusts the available

context, and requests suggestions. The GUI displays several activities to compare, save, or dismiss.

Input: Manual or approximate location, date and time, available gear, user preferences, previous sessions, available time, and maximum travel distance. Weather and lighting conditions may be retrieved automatically from external tools. Precise/current device location is used only after explicit permission.

Output: Several realistic activity suggestions, each with a subject, suitable location or setting, estimated time and travel, and a brief reason it fits.

AI Involvement: Hybrid. AI generates and explains ideas; deterministic checks validate time, distance, gear, and permission constraints using available location and external API data.

Expected Workflow: Gather permitted context → generate candidates → check feasibility → rank and display suggestions → let the user select a suggestion and pass it to the shoot-planning workflow.

Error/Alternative Cases: If location permission is denied or context is missing, use manual input or broader suggestions. If conditions make suggestions impractical, explain the constraint and offer alternatives.

F03 Natural-Language Photography Requirement Parsing

Description: Interprets a user's planned shoot. F02 answers "What should I shoot?"; F03 handles "I know what I want to shoot; understand my requirements."

GUI Interaction: The user types a description into a shoot request field, reviews the extracted requirements, answers any clarification questions, and confirms or edits the result.

Input: A natural-language description of the shoot, plus any answers to follow-up questions.

Output: A structured ShootRequirement containing subject or genre, date or time, location, duration, shooting goals, portability preferences, gear constraints, desired camera count, lens-change restrictions, and relevant priorities where provided.

AI Involvement: Hybrid. AI interprets natural-language shooting requests, extracts structured requirements, and identifies missing or ambiguous information. Deterministic validation checks required fields, accepted values, and consistency with user-confirmed constraints.

Expected Workflow: Parse the request → display extracted fields → ask concise questions about missing or ambiguous required information → incorporate answers → obtain user confirmation → attach the structured ShootRequirement to the current photography session for later planning.

Error/Alternative Cases: If the request is unclear or contradictory, highlight the affected fields and ask for clarification. If parsing fails, preserve the original text and allow manual entry.

F04 AI Gear Recognition and Automatic Import

Description: Adds a camera or lens from a photo of its markings, product label, or order screenshot without requiring the user to select a brand or model.

GUI Interaction: The user uploads an image, reviews the identified product and specifications, then confirms or corrects the match before adding it to their gear inventory.

Input: An equipment or order image; optional user correction when identification is uncertain.

Output: A user-confirmed gear entry with the identified model and available specifications, with any unverified information clearly labeled, ready to be added to the gear inventory.
AI Involvement: Hybrid. AI reads product-relevant text and proposes matches. An external retrieval tool checks the identified model and retrieves specifications from available sources, including newly released products when reliable information is available.
Expected Workflow: Upload image → extract product clues → find likely matches → verify models and specifications through retrieval → present the best match → save only after user confirmation.
Error/Alternative Cases: If the image is unreadable, lookup fails, or several models match, request a clearer image or user selection. Flag conflicting specifications for review. For order screenshots, the user will be prompted to crop or redact names, addresses, order numbers, payment details, and other personal information before external AI processing. Original screenshots will not be retained unnecessarily.

F05 Photography Gear Inventory Management

Description: Maintains a usable record of owned cameras, lenses, filters, batteries, and storage media, including items confirmed through F04. It stores specifications needed for shoot planning, gear matching, compatibility checks, and technical recommendations.
GUI Interaction: The user browses a gear list, searches or filters by type, and adds, edits, searches, filters, or removes items through desktop forms.
Input: Equipment type, brand and model where applicable, quantity, and relevant details such as camera mount, lens focal range and aperture, filter size, battery compatibility, or storage capacity.
Output: A searchable inventory of owned gear and its planning-relevant specifications.
AI Involvement: Deterministic. F05 manages confirmed records; AI recognition belongs to F04.
Expected Workflow: Add an item manually or accept a confirmed F04 import → review its details → save it → find and update it as the collection changes. Other features can read the inventory when making recommendations.
Error/Alternative Cases: Flag missing or invalid details and possible duplicates before saving. If an item is used by a saved session, warn before removal and preserve the session's historical gear record.

F06 AI Photography Equipment Capability Requirement Analysis

Description: Determines the equipment capabilities a planned shoot needs, without naming brands or products. F06 answers "What kind of equipment do I need?"; F07 later matches those needs against the user's gear.
GUI Interaction: The user opens a confirmed shoot requirement, requests capability analysis, and reviews or adjusts the resulting priorities.
Input: Confirmed ShootRequirement, shooting goals, available environment context, portability preferences, and technical priorities.
Output: An EquipmentRequirementProfile listing only relevant needs, such as focal-length range, maximum aperture, autofocus, burst rate, stabilization, resolution, weight, or storage capacity, with brief reasons and priorities.
AI Involvement: Hybrid. AI interprets the shoot's demands; deterministic checks keep the profile within valid ranges and consistent with confirmed constraints.

Expected Workflow: Read the confirmed requirement → identify shooting challenges → derive and prioritize needed capabilities → show the profile for user review → save it for F07.

Error/Alternative Cases: Ask for clarification if a missing detail would materially change the profile. Flag conflicting priorities, such as minimal weight and extensive equipment, and let the user choose which matters more.

F07 Owned Gear Matching and Compatibility Validation

Description: Compares the EquipmentRequirementProfile from F06 with the confirmed GearInventory from F05. It answers, "Which of my existing gear can satisfy these requirements?"

GUI Interaction: The user opens a shoot's equipment profile, selects Match My Gear, and reviews proposed loadouts, compatibility results, and trade-offs.

Input: The equipment requirement profile and confirmed records for owned cameras, lenses, filters, batteries, and storage media.

Output: Ranked, usable loadouts with matched requirements, unmet needs, compatibility checks, and brief reasons for each recommendation.

AI Involvement: Hybrid. Deterministic rules validate mount, filter size, battery type, and storage support. AI compares and ranks compatible loadouts based on the priorities defined in the EquipmentRequirementProfile, explaining the trade-offs behind each recommendation.

Expected Workflow: Retrieve owned gear → reject incompatible combinations → compare remaining equipment with required capabilities → rank loadouts → show limitations → let the user select a loadout for the shoot.

Error/Alternative Cases: If inventory details are missing, mark compatibility as unverified and request confirmation. If no complete loadout exists, show the closest compatible combination and its unmet requirements without presenting incompatible gear as usable.

F08 Photography Technical Calculation Tool Suite

Description: A shared suite of precise tools for depth of field, hyperfocal distance, field of view, exposure and ND adjustment, storage needs, and battery or shooting-capacity estimates. Planning features can use the same tools as the user.

GUI Interaction: The user selects a calculation, enters or confirms its parameters, and views the result with units and assumptions.

Input: Tool-specific values, such as sensor size, focal length, aperture, focus distance, exposure settings, image size, shooting duration, or battery capacity.

Output: Numerical results with units, relevant assumptions, and a clear indication when a result is an estimate.

AI Involvement: Deterministic. An AI agent may decide when to invoke a calculation tool, but the calculations themselves are performed deterministically rather than by the LLM.

Expected Workflow: Select a tool → obtain parameters from the user or confirmed shoot and gear records → validate them → calculate → return results for display or later planning.

Error/Alternative Cases: Reject invalid values, request missing parameters, and flag unsupported gear data. Where capacity depends on uncertain usage, show an estimate and its assumptions rather than a precise guarantee.

F09 Location, Time, and Environment-Aware Photography Planning

Description: Provides environmental context for a planned shoot: suitable shooting windows, suggested arrival times, and weather or lighting considerations. F02 discovers what to shoot; F09 assesses when and under what conditions; F10 builds the full shoot plan.
GUI Interaction: The user opens a planned shoot, enters a manual or approximate location and date, then reviews a timeline of suggested windows and conditions. Precise/current device location is requested only with explicit permission.
Input: Shoot location, date, intended subject, available time, and environmental data retrieved from external tools or APIs, such as weather and sunrise/sunset information.
Output: Suggested shooting windows and arrival times, with supporting conditions, source timestamps, and practical considerations.
AI Involvement: Hybrid. External tools supply factual weather and sun times; deterministic time checks and AI interpretation turn that context into shoot-specific guidance.
Expected Workflow: Obtain a permitted or manually entered location → retrieve weather and sunlight data through available APIs → validate data availability and timestamps → assess the shoot's timing and lighting needs → present recommendations for F10, clearly identifying any estimates or unavailable data.
Error/Alternative Cases: If permission is denied, use manual location. If data is unavailable or stale, label the limitation and avoid precise claims.

F10 AI Shoot Plan Generation and Interactive Modification

Description: Integrates confirmed requirements (F03), equipment needs and owned-gear matches (F06–F07), calculations (F08), and environmental context (F09) into an actionable shoot plan. F02 helps choose a subject; F10 plans how to photograph it.
GUI Interaction: The user selects Generate Plan, reviews the proposed plan, and requests changes in natural language, such as “make it lighter” or “prioritize close-ups.”
Input: Confirmed ShootRequirement, EquipmentRequirementProfile, compatible owned loadouts, available calculation results, environmental context, and later modification requests.
Output: A saved plan with loadout and camera/lens assignments, shooting settings or strategies, timing, preparation notes, packing checklist, and shot priorities.
AI Involvement: Hybrid. AI combines evidence and revises the plan; deterministic checks validate gear compatibility, numerical results, and fixed constraints.
Expected Workflow: Assemble inputs → generate a plan → validate it → present it for review → apply requested changes while preserving unaffected constraints → save the approved revision.
Error/Alternative Cases: Ask for clarification when a change is ambiguous or conflicts with a fixed constraint. If required data is missing or stale, identify the gap and mark affected recommendations as provisional.

F11 AI Post-Shoot Analysis and Review

Description: Reviews a completed session using imported photos or EXIF metadata. It compares recorded shooting choices with the original ShootPlan, user goals, and previous sessions to identify technical and workflow lessons.
GUI Interaction: The user opens a completed session, imports photos or metadata, reviews the statistics, and requests a written review.
Input: Supported photographs (initially JPEG) or exported EXIF metadata, the saved

ShootPlan, stated goals, and available session history. Additional image formats may be supported where metadata extraction is available.

Output: A summary of settings and camera/lens usage, observed patterns, plan-versus-actual differences, and practical recommendations for similar future shoots.

AI Involvement: Hybrid. Deterministic processing extracts EXIF fields and calculates statistics for focal length, shutter speed, aperture, ISO, and equipment use. AI interprets those verified results in context; it does not invent measurements or score aesthetics.

Expected Workflow: Import → extract and validate metadata → calculate session statistics → compare them with the plan and history → generate a review → save it with the session.

Error/Alternative Cases: Report unreadable files or missing EXIF fields, analyze the available data, and label conclusions limited by incomplete metadata.

3. UML Class Diagram

Attached as files in PNG and UXF formats named "PhotoPilot-ClassDiagram."

4. Design Pattern Explanations

4.1 Facade Pattern

PhotoPilot requires its GUI and CLI to provide consistent access to session management, equipment workflows, planning, calculations, and review. Without a shared entry point, both interfaces would need to coordinate several application services independently.

PhotoPilotFacade provides this entry point. Its clients are PhotoPilotGUI and PhotoPilotCLI. Its collaborating subsystem classes are SessionService, InventoryService, DiscoveryService, PlanningService, ReviewService, and TechnicalCalculator. The Facade delegates requests while these classes retain their respective workflow and domain responsibilities.

For example, createSession() delegates session creation to SessionService, generatePlan() delegates planning to PlanningService.generate(), and calculate() uses TechnicalCalculator.calculate(). Both presentation interfaces therefore access the same underlying functionality. The UML also exposes services through sessions(), inventory(), planning(), and reviews(), so the Facade simplifies access without completely hiding the subsystem. This is consistent with the lecture's description of Facade.

The pattern supports separation of presentation and application responsibilities and reduces duplicated coordination. Without it, changes to service organization could require corresponding changes in both interfaces, increasing coupling and the risk of inconsistent behavior.

Related features: F01–F11, through the shared application entry points.

4.2 Strategy Pattern

F07 must compare compatible equipment combinations according to different photographic priorities. A photographer may prefer lower carrying weight in one session and broader focal-length coverage in another. Embedding every ranking approach inside the matching workflow would couple compatibility processing to changing recommendation policies.

GearMatchingService is the Context, and LoadoutRankingStrategy defines the Strategy interface through rank(candidates, profile). PortabilityRanking and CoverageRanking are the Concrete Strategies. Both reference PhotoPilotAgent, but emphasize different priorities

when comparing loadouts.

GearMatchingService retains a LoadoutRankingStrategy reference and provides setRanking() to replace the selected strategy. Its matching workflow uses CompatibilityValidator to establish compatible candidates before invoking ranking. The diagram also specifies validation of returned candidate identifiers, preventing AI ranking from introducing unverified equipment combinations.

This structure follows the lecture's definition of interchangeable algorithms accessed through a common interface. Ranking behavior can vary without rewriting the deterministic compatibility rules. The interface also permits controlled substitutes when testing the matching workflow.

Without Strategy, portability and coverage decisions would accumulate as conditional branches within GearMatchingService, making ranking changes harder to isolate and maintain.

Related features: F07 directly; F10 subsequently consumes the recommended loadouts.

4.3 Adapter Pattern

PhotoPilot integrates external systems whose request formats and results differ from its application interfaces. Adapter isolates these differences so application services can work with PhotoPilot-defined contracts.

For runtime AI, PhotoPilotAgent is the Client, LLMClient is the Target interface, and GeminiAdapter is the Adapter. The external Google Gemini API is the adapted service, accessed through the adapter's HttpClient and configured endpoint. complete(context) converts between AgentContext and the integration's request/response representation, returning Result<ModelReply>.

For metadata processing, ReviewService is the Client, MetadataExtractor is the Target, and ExifLibraryAdapter implements supports(file) and extract(files). The intended adaptee is an EXIF-processing library, although its concrete type and delegation relationship are not specified in the UML. EnvironmentApiAdapter and EquipmentCatalogAdapter similarly implement application-defined integration interfaces.

These boundaries support dependency inversion and allow integration failures or responses to be tested through substitute implementations. External representation changes remain concentrated in adapters. Without them, agent and review workflows would become coupled to provider-specific formats and library details.

Related features: F02, F03, F04, F06, F07, F09, F10, and F11 through their respective AI, environmental, equipment-retrieval, or metadata integrations.

4.4 Command Pattern

F10 accepts natural-language modifications while requiring compatibility checks and preservation of fixed constraints. Representing each modification as an object separates the requested change from the mechanism that applies and validates it.

PlanCommand is the Command interface, declaring applyTo(draft). ReplaceLoadoutCommand, ReviseScheduleCommand, and ReviseStrategyCommand are Concrete Commands containing their modification data. PlanEditor is the Invoker, ShootPlan is the Receiver, and PlanningService coordinates the application workflow through methods including proposeChange() and approve().

PlanEditor.preview() and applyApproved() accept a PlanCommand. The receiver supports

copyForRevision(), allowing changes to operate on a draft rather than overwrite the saved base. Concrete commands use the corresponding receiver operations: replaceLoadout(), reviseSchedule(), or reviseStrategy(). PlanValidator supplies deterministic validation, including validateChange(), to enforce the established constraints.

This follows the lecture's principle of encapsulating requests as objects passed to an invoker. It makes modification behavior independently understandable and testable while keeping validation centralized. Without Command, planning or presentation code would need separate mutation logic for each change category.

The design supports controlled revisions; it does not specify undo/redo.

Related features: F10 directly, using equipment and timing information established by earlier features.

4.5 State Pattern

A photography session has different permitted behavior during preparation, planning, completion, and review. Distributing lifecycle checks throughout application services would make these rules difficult to maintain consistently.

PhotographySession is the Context and holds a SessionState reference. SessionState defines next(session, event) and canEditPlan(). The Concrete States are DraftState, PlannedState, CompletedState, and ReviewedState, each implementing this interface.

SessionService.transition() provides an application-level transition operation, while PhotographySession.apply(event) provides the context operation through which state-dependent transition behavior is invoked. The current state's next() returns a Result<SessionState>, allowing a transition request to succeed or report failure. The UML identifies approval of a valid plan as the transition from Draft to Planned, shoot completion from Planned to Completed, and a saved review from Completed to Reviewed. These correspond to the explicit SessionEvent values; they are not automatic time-based transitions.

The pattern localizes lifecycle behavior and supports testing each state's transition and editing rules. Unlike a status enum alone, it supplies polymorphic behavior. Without it, repeated conditionals could permit inconsistent transitions or inappropriate changes to historical plans.

Related features: F01 directly, with lifecycle constraints supporting F10 and F11.

5. Use-Case Diagram

Attached as files in PNG and UXF formats named "PhotoPilot-UseCaseDiagram."

6. Detailed Use-Case Descriptions

The following descriptions apply to PhotoPilot's GUI and CLI through their shared application services. External services participate only where identified. UC14 is included by UC07, UC10, and UC11; it has no independent user interaction.

6.1 UC01 — Manage sessions and search history

Use Case ID: UC01

Use Case Name: Manage sessions and search history

Actor(s): Photographer; Device Location Service, optionally.

Goal: Maintain retrievable photography sessions and their associated records.

Preconditions: PhotoPilot is running.
Trigger: The photographer requests session creation or searches session history.
Main Success Scenario:
1. The photographer enters a title, date, and optional location, notes, or goals.
2. PhotoPilot validates the details.
3. PhotoPilot saves the session with a unique identifier, status, and timestamps.
4. The photographer searches history and selects a saved session.
5. PhotoPilot restores its details and linked plans, statistics, and reviews.
Alternative/Exception Flows:
A1 — Step 1: Device location is requested only after explicit permission; otherwise, manual location or omission is available.
A2 — Step 4: No matches produce an empty-results message.
E1 — Steps 2–5: Invalid details or persistence failures are reported; unsaved entries are preserved.
Postconditions: Successful records remain retrievable. Failed operations are not reported as saved or loaded.
Related Feature(s): F01.

6.2 UC02 — Discover photography activities

Use Case ID: UC02
Use Case Name: Discover photography activities
Actor(s): Photographer; Google Gemini API; Weather / Sunlight Data Service and Device Location Service, when used.
Goal: Identify feasible activities matching the photographer's circumstances.
Preconditions: The photographer can provide or review available context.
Trigger: The photographer requests photography suggestions.
Main Success Scenario:
1. The photographer specifies location, time, preferences, available gear, and travel constraints.
2. PhotoPilot gathers permitted context, available history, and relevant environmental data.
3. Gemini proposes activities; deterministic checks assess their feasibility.
4. PhotoPilot presents ranked suggestions with subjects, settings, estimates, and reasons.
5. The photographer selects a suggestion for subsequent requirement confirmation.
Alternative/Exception Flows:
A1 — Step 2: Denied location permission or missing context leads to manual input or broader suggestions.
A2 — Step 5: The photographer may save a suggestion to a session or dismiss it.
E1 — Steps 2–3: Unavailable environmental data is identified; Gemini failure produces an error without fabricated recommendations.
Postconditions: Suggestions are displayed; selection does not constitute confirmed requirements.
Related Feature(s): F02.

6.3 UC03 — Interpret and confirm shoot requirements

Use Case ID: UC03
Use Case Name: Interpret and confirm shoot requirements

Actor(s): Photographer; Google Gemini API.
Goal: Establish a validated, user-confirmed interpretation of the intended shoot.
Preconditions: A photography session is open.
Trigger: The photographer submits a natural-language shooting request.
Main Success Scenario:
1. The photographer describes the subject, goals, timing, and relevant constraints.
2. PhotoPilot uses Gemini to produce structured requirements.
3. PhotoPilot displays the extracted fields and requests necessary clarification.
4. The photographer answers questions and corrects the interpretation.
5. PhotoPilot validates required fields, accepted values, and constraint consistency.
6. The photographer confirms the result; PhotoPilot attaches it to the session.
Alternative/Exception Flows:
A1 — Steps 3–5: Ambiguous or contradictory fields are highlighted for clarification.
E1 — Step 2: Parsing failure preserves the original text and permits manual entry followed by validation.
A2 — Step 6: Without confirmation, the draft is not treated as confirmed.
Postconditions: Successful completion establishes confirmed requirements for later planning.
Related Feature(s): F03.

6.4 UC04 — Recognize gear from an image and confirm import

Use Case ID: UC04
Use Case Name: Recognize gear from an image and confirm import
Actor(s): Photographer; Google Gemini API; External Equipment Specification Service.
Goal: Add correctly identified equipment using an image.
Preconditions: An equipment image or order screenshot is available.
Trigger: The photographer requests image-based equipment import.
Main Success Scenario:
1. The photographer supplies an image and confirms any necessary cropping or redaction before external processing.
2. Gemini extracts product clues and proposes equipment matches.
3. PhotoPilot retrieves model specifications from the external service.
4. PhotoPilot presents the proposed match, specifications, and uncertainties.
5. The photographer corrects or confirms the entry.
6. PhotoPilot validates and saves the confirmed equipment.
Alternative/Exception Flows:
A1 — Steps 2–4: Unreadable images or multiple matches prompt a clearer image or user selection.
E1 — Steps 2–3: Service failures are reported; missing or conflicting specifications remain unverified.
A2 — Step 5: Cancellation prevents import.
Postconditions: Only confirmed, validated equipment is saved. Original screenshots are not retained unnecessarily.
Related Feature(s): F04; F05 through the resulting inventory entry.

6.5 UC05 — Manage owned gear inventory

Use Case ID: UC05
Use Case Name: Manage owned gear inventory
Actor(s): Photographer.
Goal: Maintain accurate, searchable records of owned equipment.
Preconditions: PhotoPilot is running.
Trigger: The photographer opens the inventory or requests a change.
Main Success Scenario:
1. PhotoPilot displays the inventory.
2. The photographer searches or filters equipment and selects an entry to edit or adds one manually.
3. The photographer supplies equipment type, quantity, and relevant specifications.
4. PhotoPilot validates the details and identifies possible duplicates.
5. The photographer reviews the entry; PhotoPilot saves it and updates the displayed inventory.
Alternative/Exception Flows:
A1 — Step 2: On removal, PhotoPilot warns about affected saved sessions and requires acceptance before proceeding; historical gear records are preserved.
E1 — Step 4: Invalid details require correction; possible duplicates are flagged for review.
E2 — Step 5: Persistence failure is reported without claiming success.
Postconditions: Successful changes are available to subsequent planning. Inventory management remains usable without Gemini.
Related Feature(s): F05.

6.6 UC06 — Determine equipment capability needs

Use Case ID: UC06
Use Case Name: Determine equipment capability needs
Actor(s): Photographer; Google Gemini API.
Goal: Determine required equipment capabilities without selecting brands or products.
Preconditions: The session contains confirmed shooting requirements.
Trigger: The photographer requests capability analysis.
Main Success Scenario:
1. PhotoPilot retrieves confirmed requirements, priorities, and available environmental context.
2. Gemini derives relevant capability needs and explains their priorities.
3. PhotoPilot checks valid ranges and consistency with confirmed constraints.
4. PhotoPilot presents the capability profile.
5. The photographer reviews or adjusts priorities.
6. PhotoPilot validates and saves the reviewed profile for gear matching.
Alternative/Exception Flows:
A1 — Steps 2–5: Materially missing information or conflicting priorities prompts clarification.
E1 — Step 2: Gemini failure is reported without presenting unsupported analysis.
E2 — Steps 3 or 6: Invalid or contradictory values must be corrected before acceptance.
Postconditions: A reviewed capability profile is available for matching. No particular equipment product has been selected.

Related Feature(s): F06.

6.7 UC07 — Match and select compatible owned gear

Use Case ID: UC07
Use Case Name: Match and select compatible owned gear
Actor(s): Photographer; Google Gemini API.
Goal: Select usable owned equipment that best satisfies the capability profile.
Preconditions: A reviewed capability profile and confirmed inventory records are available.
Trigger: The photographer selects Match My Gear.
Main Success Scenario:
1. PhotoPilot retrieves the profile and owned equipment.
2. PhotoPilot invokes UC14 for candidate combinations, excluding incompatible or unverified combinations from usable recommendations.
3. PhotoPilot compares compatible candidates with required capabilities.
4. Gemini ranks those candidates and explains trade-offs; PhotoPilot verifies that ranking introduces no unvalidated candidates.
5. The photographer reviews limitations and selects a loadout.
Alternative/Exception Flows:
A1 — Step 2: Missing specifications prompt confirmation.
A2 — Step 3: If no complete match exists, available compatible partial matches show unmet needs.
E1 — Step 4: Gemini failure prevents AI ranking but does not invalidate deterministic compatibility results.
Postconditions: A compatible loadout is selected, or limitations are reported. AI ranking never overrides compatibility constraints.
Related Feature(s): F07.

6.8 UC08 — Calculate photography settings and capacity

Use Case ID: UC08
Use Case Name: Calculate photography settings and capacity
Actor(s): Photographer.
Goal: Obtain numerical photography results with explicit units and assumptions.
Preconditions: PhotoPilot is running.
Trigger: The photographer selects a technical calculation.
Main Success Scenario:
1. The photographer selects depth of field, hyperfocal distance, field of view, exposure/ND adjustment, storage, or battery/shooting-capacity calculation.
2. PhotoPilot obtains parameters from user input or confirmed session and gear records.
3. The photographer reviews the parameters.
4. PhotoPilot validates them and performs the deterministic calculation.
5. PhotoPilot displays the result, units, assumptions, and applicable estimate labels.
Alternative/Exception Flows:
E1 — Steps 2–4: Missing, invalid, or unsupported values require correction before calculation.
A1 — Step 5: Capacity results dependent on uncertain usage are explicitly identified as estimates.

Postconditions: A valid result is available for display or later planning; rejected inputs produce no purported valid result. Gemini availability does not affect this use case.
Related Feature(s): F08.

6.9 UC09 — Assess shooting windows and environmental conditions

Use Case ID: UC09
Use Case Name: Assess shooting windows and environmental conditions
Actor(s): Photographer; Google Gemini API; Weather / Sunlight Data Service; Device Location Service, optionally.
Goal: Identify suitable shooting windows, arrival times, and environmental considerations.
Preconditions: A session provides the intended subject and timing context.
Trigger: The photographer requests environmental assessment.
Main Success Scenario:
1. The photographer confirms location, date, and available time.
2. PhotoPilot retrieves weather and sunlight data.
3. PhotoPilot checks availability, timestamps, and timing consistency.
4. Gemini interprets verified context for the intended shoot.
5. PhotoPilot presents shooting windows, arrival suggestions, supporting conditions, and limitations.
Alternative/Exception Flows:
A1 — Step 1: Device location requires permission; denial or unavailability permits manual input.
E1 — Steps 2–3: Missing or stale data is identified; affected guidance avoids unsupported precision.
E2 — Step 4: Gemini failure is reported; factual retrieval remains independent of AI interpretation.
Postconditions: Available assessment results support planning, with estimates and unavailable evidence identified.
Related Feature(s): F09.

6.10 UC10 — Generate and approve a shoot plan

Use Case ID: UC10
Use Case Name: Generate and approve a shoot plan
Actor(s): Photographer; Google Gemini API.
Goal: Obtain an approved, actionable shooting plan.
Preconditions: A session permits planning and contains confirmed requirements and a reviewed capability profile.
Trigger: The photographer requests plan generation.
Main Success Scenario:
1. PhotoPilot assembles requirements, gear matches, available calculations, environmental context, and relevant history.
2. Gemini proposes equipment assignments, settings, timing, preparation notes, a packing checklist, and shot priorities.
3. PhotoPilot invokes UC14 and validates numerical results and fixed constraints.
4. PhotoPilot presents the validated proposal.
5. The photographer approves it.

6. PhotoPilot saves the plan; initial valid approval changes a Draft session to Planned.
Alternative/Exception Flows:
A1 — Step 1: Missing or stale evidence is identified; affected recommendations are provisional.
E1 — Steps 2–3: API failure or failed validation prevents acceptance of an unsupported plan.
A2 — Step 5: Unapproved proposals are not saved as approved plans.
Postconditions: Successful approval produces a retrievable plan.
Related Feature(s): F10.

6.11 UC11 — Revise and approve a shoot plan

Use Case ID: UC11
Use Case Name: Revise and approve a shoot plan
Actor(s): Photographer; Google Gemini API.
Goal: Modify a plan while preserving fixed constraints and unaffected decisions.
Preconditions: A saved plan exists, and the session permits plan editing.
Trigger: The photographer submits a natural-language modification request.
Main Success Scenario:
1. PhotoPilot retrieves the saved plan and confirmed constraints.
2. Gemini interprets the requested change.
3. PhotoPilot prepares a revised draft without overwriting the saved plan.
4. PhotoPilot invokes UC14 and validates calculations and constraint preservation.
5. The photographer reviews and approves the revision.
6. PhotoPilot saves the approved revision with the session.
Alternative/Exception Flows:
A1 — Step 2: Ambiguity prompts clarification.
E1 — Step 4: Incompatibility or conflict with fixed constraints prevents approval.
E2 — Steps 2 or 6: API or save failures are reported without claiming a completed revision.
A2 — Step 5: Rejection leaves the saved plan unchanged.
Postconditions: A validated revision is saved only after approval; otherwise, the saved base remains unchanged.
Related Feature(s): F10.

6.12 UC12 — Import metadata and view shooting statistics

Use Case ID: UC12
Use Case Name: Import metadata and view shooting statistics
Actor(s): Photographer.
Goal: Obtain objective statistics from recorded shooting metadata.
Preconditions: A completed session is open, and supported photographs or exported EXIF metadata are available.
Trigger: The photographer requests metadata import and analysis.
Main Success Scenario:
1. The photographer selects supported files, initially JPEG photographs or exported EXIF metadata.
2. PhotoPilot extracts and validates available metadata.
3. PhotoPilot deterministically calculates focal-length, shutter-speed, aperture, ISO, and

equipment-use statistics.
4. PhotoPilot displays the statistics and identifies incomplete evidence.
5. PhotoPilot associates the analysis with the session for retrieval and review.
Alternative/Exception Flows:
E1 — Step 2: Unsupported or unreadable files are reported; usable records remain analyzable.
A1 — Steps 2–3: Missing fields are excluded from affected calculations and reported.
E2 — Step 3: No usable metadata produces an explanation rather than invented statistics.
Postconditions: Available statistics reflect extracted evidence only. Analysis remains functional without Gemini and does not assess artistic quality.
Related Feature(s): F11.

6.13 UC13 — Obtain and save a post-shoot review

Use Case ID: UC13
Use Case Name: Obtain and save a post-shoot review
Actor(s): Photographer; Google Gemini API.
Goal: Derive technical and workflow lessons from a completed shoot.
Preconditions: A completed session has usable metadata statistics, a saved plan, and stated goals.
Trigger: The photographer requests a written review.
Main Success Scenario:
1. PhotoPilot retrieves statistics, the plan, goals, and available session history.
2. PhotoPilot derives evidence-based comparisons between planned and recorded shooting choices.
3. Gemini interprets those results and proposes practical lessons.
4. PhotoPilot presents the review with evidence limitations.
5. The photographer requests saving; PhotoPilot saves the review with the session and changes Completed to Reviewed.
Alternative/Exception Flows:
A1 — Steps 1–4: Missing metadata or history limits conclusions and is explicitly acknowledged.
E1 — Step 3: Gemini failure leaves deterministic statistics available without fabricating a review.
E2 — Step 5: Save failure is reported without claiming successful persistence.
Postconditions: A saved review supports future retrieval. It neither invents measurements nor scores aesthetics.
Related Feature(s): F11.

6.14 UC14 — Validate gear compatibility

Use Case ID: UC14
Use Case Name: Validate gear compatibility
Actor(s): No directly associated external actor; supports the Photographer through UC07, UC10, and UC11.
Goal: Establish whether proposed equipment combinations satisfy compatibility rules.
Preconditions: An including use case supplies a candidate loadout and available equipment specifications.

Trigger: UC07, UC10, or UC11 invokes its mandatory <<include>> behavior.
Main Success Scenario:
1. PhotoPilot examines the supplied equipment and specifications.
2. Deterministic rules check camera–lens mounts, filter sizes, battery compatibility, and storage support where applicable.
3. PhotoPilot records compatibility results, violations, and unverified information.
4. PhotoPilot returns the report to the including use case.
Alternative/Exception Flows:
A1 — Step 2: A detected mismatch produces an incompatible result.
E1 — Steps 1–2: Missing or unsupported specifications produce an unverified result, never presumed compatibility.
A2 — Step 4: The including use case excludes unusable candidates or requests correction.
Postconditions: A compatibility report is returned without changing inventory or approving a plan. An incompatible result is a completed validation outcome, not a successful compatibility finding.
Related Feature(s): F07; F10 through plan generation and revision.

7. Sequence Diagrams

Attached as files in PNG and UXF formats named "PhotoPilot-SD01-05."

8. Feature-to-Design Traceability Table

Feature ID	Description	Type	Related Use Case(s)	Related Classes	Key Methods	Sequence Diagram(s)	Design Pattern(s)
F01	Create, save, search and reopen photography sessions and linked history.	Deterministic	UC01	PhotoPilotGUI; PhotoPilotCLI; PhotoPilotFacade; SessionService; PhotographySession; SessionRepository; JsonSessionRepository; SessionState; DraftState; PlannedState; CompletedState; ReviewedState	PhotoPilotFacade: createSession(title, date); sessions() SessionService: create(title, date); save(session); search(query); open(id); transition(id, event) PhotographySession: apply(event) SessionRepository: save(session); find(id); search(query)	SD01: creation, history and completion. Gap: optional location/notes have no exposed mutation route.	Facade: shared access. State: session lifecycle.
F02	Suggest feasible shooting activities from permitted context; support choosing, saving or dismissing ideas.	Hybrid	UC02	PhotoPilotGUI; PhotoPilotCLI; PhotoPilotFacade; DiscoveryService; DiscoveryContext; ActivitySuggestion; FeasibilityValidator; EnvironmentService; EnvironmentProvider; EnvironmentApiAdapter; SessionRepository; PhotoPilotAgent; LLMClient; GeminiAdapter	PhotoPilotFacade: suggest(context) DiscoveryService: suggest(context); select(id, idea); save(id, idea); dismiss(idea) FeasibilityValidator: validate(idea, context) EnvironmentService: retrieve(place, date)	SD01: suggestions and feasibility. Gap: select/save/dismiss lack a Facade route and are not sequenced.	Facade: suggestions. Adapter: Gemini and environment data.
F03	Parse a brief, clarify or edit fields, validate and save confirmed shoot requirements.	Hybrid	UC03	PhotoPilotGUI; PhotoPilotCLI; PhotoPilotFacade; PlanningService; RequirementService; ShootRequirement; PhotographySession; SessionRepository; PhotoPilotAgent; LLMClient; GeminiAdapter	PhotoPilotFacade: parse(id, text); planning() PlanningService: parse(id, text); confirm(id, edited) RequirementService: parse(text, context); clarify(draft, answers); confirm(edited) ShootRequirement: validate(); confirm() PhotographySession: attachRequirement(requirement)	SD01: parsing and explicit confirmation; clarification via resubmitted text.	Facade: shared workflow. Adapter: Gemini parsing.

Feature ID	Description	Type	Related Use Case(s)	Related Classes	Key Methods	Sequence Diagram(s)	Design Pattern(s)
F04	Recognize gear from an approved, redacted image; verify specifications and import only after confirmation.	Hybrid	UC04	PhotoPilotGUI; PhotoPilotCLI; PhotoPilotFacade; ImagePrivacyBoundary; SanitizedImage; InventoryService; GearRecognitionService; GearImportDraft; Equipment; EquipmentSpecificationProvider; EquipmentCatalogAdapter; InventoryRepository; PhotoPilotAgent; LLMClient; GeminiAdapter	ImagePrivacyBoundary: approveRedacted(copy, confirmed); discard(image) PhotoPilotFacade: recognize(image); inventory() GearRecognitionService: recognize(image); verify(draft); confirm(edited) EquipmentSpecificationProvider: lookup(model) InventoryService: confirm(draft); save(item)	SD02: privacy, recognition, lookup and confirmed import.	Facade: import access. Adapter: Gemini and equipment lookup.
F05	Maintain confirmed gear records; validate entries, flag duplicates and warn before removal while preserving history.	Deterministic	UC05; UC04 (resulting inventory entry)	PhotoPilotGUI; PhotoPilotCLI; PhotoPilotFacade; InventoryService; GearInventory; Equipment; Camera; Lens; Filter; Battery; StorageMedia; InventoryRepository; JsonInventoryRepository; SessionRepository; GearSnapshot	PhotoPilotFacade: saveGear(item); inventory() InventoryService: list(query); save(item); duplicates(item); affectedSessions(id); remove(id, warningAccepted) GearInventory: add(item); update(item); search(query); remove(id) SessionRepository: referencingGear(id)	SD02: inventory operations and confirmed-import persistence.	Facade: common inventory access.
F06	Derive capability needs without product names; validate, review and save the confirmed profile.	Hybrid	UC06	PhotoPilotGUI; PhotoPilotCLI; PhotoPilotFacade; PlanningService; RequirementService; ShootRequirement; EquipmentRequirementProfile; CapabilityNeed; PhotographySession; SessionRepository; PhotoPilotAgent; LLMClient; GeminiAdapter	PhotoPilotFacade: analyzeNeeds(id); planning() PlanningService: analyzeNeeds(id); confirmProfile(id, edited) RequirementService: deriveProfile(req) EquipmentRequirementProfile: validate(req); confirm() PhotographySession: attachProfile(profile)	SD02: analysis plus revalidated profile confirmation and atomic save.	Facade: shared workflow. Adapter: Gemini analysis.

Feature ID	Description	Type	Related Use Case(s)	Related Classes	Key Methods	Sequence Diagram(s)	Design Pattern(s)
F07	Check owned-gear compatibility, match capability needs and rank verified usable loadouts with trade-offs.	Hybrid	UC07; UC14 (included)	PhotoPilotGUI; PhotoPilotCLI; PhotoPilotFacade; PlanningService; GearMatchingService; CompatibilityValidator; CompatibilityReport; LoadoutRankingStrategy; PortabilityRanking; CoverageRanking; EquipmentRequirementProfile; GearInventory; Loadout; PhotoPilotAgent; LLMClient; GeminiAdapter	PhotoPilotFacade: matchGear(id) GearMatchingService: match(profile, gear); compatibleCandidates(gear); setRanking(strategy) CompatibilityValidator: validate(loadout); matchNeeds(loadout, profile) CompatibilityReport: isUsable() LoadoutRankingStrategy: rank(candidates, profile)	SD03: compatibility and ranking. Gap: no operation attaches a selected loadout; ranking strategy is assumed configured.	Facade: matching access. Strategy: ranking policy. Adapter: Gemini ranking.
F08	Calculate depth of field, hyperfocal distance, field of view, exposure/ND, storage and battery estimates with units.	Deterministic	UC08	PhotoPilotGUI; PhotoPilotCLI; PhotoPilotFacade; TechnicalCalculator; CalculationInput; CalculationResult; CalculationTool	PhotoPilotFacade: calculate(input) CalculationInput: validate() TechnicalCalculator: calculate(input); depthOfField(input); hyperfocal(input); fieldOfView(input); exposureAndND(input); storage(input); batteryCapacity(input) CalculationTool: execute(argumentsJson)	SD03: all six calculations. SD04: agent invocation through CalculationTool.	Facade: direct calculation access. Tool invocation does not make calculation AI-based.

Feature ID	Description	Type	Related Use Case(s)	Related Classes	Key Methods	Sequence Diagram(s)	Design Pattern(s)
F09	Retrieve weather/sunlight evidence and recommend shooting windows and arrival times, labeling stale or missing data.	Hybrid	UC09	PhotoPilotGUI; PhotoPilotCLI; PhotoPilotFacade; PlanningService; EnvironmentService; EnvironmentProvider; EnvironmentApiAdapter; LocationService; DeviceLocationProvider; EnvironmentContext; EnvironmentAssessment; ShootingWindow; FeasibilityValidator; PhotoPilotAgent; LLMClient; GeminiAdapter	PhotoPilotFacade: assessEnvironment(id) EnvironmentService: assess(req); retrieve(place, date) EnvironmentProvider: fetch(place, date) LocationService: manual(label, lat, lon); current(explicitConsent) EnvironmentContext: staleFields(now, maxAge) FeasibilityValidator: checkTiming(assessment)	SD03: consent/manual location, retrieval, AI assessment and timing validation.	Facade: assessment access. Adapter: environmental data and Gemini.
F10	Generate, validate and preview a shoot plan; apply controlled revisions and save only the approved, revalidated version.	Hybrid	UC10; UC11; UC14 (included by both)	PhotoPilotGUI; PhotoPilotCLI; PhotoPilotFacade; PlanningService; PhotoPilotAgent; LLMClient; GeminiAdapter; HistoryRetriever; ToolRegistry; CalculationTool; GearMatchingService; PlanEditor; PlanCommand; ReplaceLoadoutCommand; ReviseScheduleCommand; ReviseStrategyCommand; ShootPlan; PlanValidator; CompatibilityValidator; TechnicalCalculator; PhotographySession; SessionState; DraftState; PlannedState; SessionRepository	PlanningService: generate(id); proposeChange(id, text); approve(id, plan) PhotoPilotAgent: run(task, context, type) ToolRegistry: execute(call) PlanEditor: preview(base, change) PlanCommand: applyTo(draft) ShootPlan: copyForRevision() PlanValidator: validate(plan, req); validateChange(base, next) PhotographySession: appendApprovedPlan(plan); apply(event) SessionRepository: save(session)	SD04: generation, tool loop, loadout-change preview and approval. Schedule/strategy commands are not separately sequenced.	Facade; Adapter (Gemini); Command (revision); State (approval). Strategy indirectly via matched loadouts.

Feature ID	Description	Type	Related Use Case(s)	Related Classes	Key Methods	Sequence Diagram(s)	Design Pattern(s)
F11	Import metadata, compute objective statistics, compare with the plan and save an evidence-based AI review; no aesthetic scoring.	Hybrid	UC12; UC13	PhotoPilotGUI; PhotoPilotCLI; PhotoPilotFacade; ReviewService; MetadataExtractor; ExifLibraryAdapter; ExifRecord; ExifStatistics; StatisticsCalculator; PostShootReview; HistoryRetriever; PhotographySession; SessionState; CompletedState; ReviewedState; SessionRepository; PhotoPilotAgent; LLMClient; GeminiAdapter	PhotoPilotFacade: analyze(id, files); review(id); reviews() ReviewService: analyze(id, files); review(id); save(id, review) MetadataExtractor: supports(file); extract(files) StatisticsCalculator: summarize(records); compare(stats, plan) PhotographySession: attachStatistics(stats); appendReview(review); apply(event) SessionRepository: save(session)	SD05: independent statistics/review requests; statistics persistence and saved-review state transition.	Facade: review access. Adapter: EXIF and Gemini. State: Completed to Reviewed.

9. Feature Implementation Explanations

9.1 F01 — Photography Session and History Management

For UC01, PhotoPilotGUI and PhotoPilotCLI would access session operations through PhotoPilotFacade. As illustrated in SD01, createSession(title, date) delegates to SessionService.create(title, date), which validates required details and initializes a PhotographySession with its identity, timestamps, and initial state. SessionRepository.save(session), with JsonSessionRepository as the proposed implementation, persists the aggregate.

For history retrieval, the client obtains SessionService through PhotoPilotFacade.sessions(). search(query) delegates to the repository, while open(id) uses SessionRepository.find(id) to restore the session and its linked records. These operations are deterministic and do not require Gemini.

The State pattern governs lifecycle changes. SessionService.transition(id, event) loads the session and invokes PhotographySession.apply(event); the current SessionState determines whether the transition is permitted. SD01 illustrates marking a planned shoot completed and saving the resulting aggregate. Invalid details, rejected transitions, and persistence failures produce unsuccessful results rather than false success messages. Empty searches return an empty history result.

9.2 F02 — AI Photography Inspiration and Activity Recommendation

For UC02, either presentation interface would submit a DiscoveryContext through PhotoPilotFacade.suggest(context). In SD01, the Facade delegates to DiscoveryService.suggest(context). The service retrieves available history through SessionRepository.search(query) and obtains environmental evidence through EnvironmentService.retrieve(place, date) when suitable context is available.

PhotoPilotAgent.run(task, context, type) requests activity suggestions through LLMClient.complete(context), implemented by GeminiAdapter. Gemini proposes and ranks activities, while FeasibilityValidator.validate(idea, context) deterministically checks each candidate against time, distance, equipment, and consent constraints. The service returns a Result<List<ActivitySuggestion>> containing supported suggestions or explicit errors and limitations.

Manual or approximate location permits discovery without device-location access; precise device location requires permission. Missing environmental evidence must remain identified, and Gemini failure must not produce fabricated recommendations. DiscoveryService declares select(id, idea), save(id, idea), and dismiss(idea), but their presentation route is unresolved. SD01 therefore demonstrates suggestion generation and feasibility checking without claiming automatic selection transfer or persistence.

9.3 F03 — Natural-Language Photography Requirement Parsing

For UC03, the presentation interface would submit a brief through PhotoPilotFacade.parse(id, text). SD01 shows delegation to PlanningService.parse(id, text), which retrieves the session using SessionRepository.find(id) and invokes RequirementService.parse(text, context). PhotoPilotAgent uses the Gemini adapter to interpret the request and return a structured ShootRequirement with any clarification questions.

Gemini performs language interpretation; deterministic domain validation checks required fields, accepted values, and consistency. Although RequirementService.clarify(draft, answers) exists, SD01 illustrates clarification by submitting revised text containing the answers.

Confirmation is a separate request through PhotoPilotFacade.planning() and PlanningService.confirm(id, edited). The service invokes RequirementService.confirm(edited) and the requirement's confirmation and validation operations. After successful validation, PhotographySession.attachRequirement(requirement) associates the confirmed result with the session, and SessionRepository.save(session) persists it.

Parsing failure preserves the original text and permits manual entry followed by confirmation. Unconfirmed drafts remain unconfirmed, and validation or save failures must not be reported as successfully persisted requirements.

9.4 F04 — AI Gear Recognition and Automatic Import

For UC04, the GUI or CLI would first use ImagePrivacyBoundary.approveRedacted(copy, confirmed) to obtain a user-approved SanitizedImage. SD02 then routes PhotoPilotFacade.recognize(image) through InventoryService.recognize(image) to GearRecognitionService.recognize(image).

PhotoPilotAgent uses Gemini to interpret product clues and propose a GearImportDraft. GearRecognitionService.verify(draft) checks candidate models through EquipmentSpecificationProvider.lookup(model), implemented by EquipmentCatalogAdapter. Retrieved specifications retain their provenance, conflicts, and unknown fields; recognition alone does not establish verified equipment data.

The user reviews and corrects the draft before a separate confirmation request. Through PhotoPilotFacade.inventory(), InventoryService.confirm(draft) delegates recognition confirmation and passes the resulting Equipment to save(item). Deterministic validation and duplicate checks precede adding the item to GearInventory and saving through InventoryRepository.

Unreadable images, ambiguous matches, unavailable services, or conflicting specifications require clarification or explicit limitations. Cancellation prevents import, and failed writes do not constitute successful additions. ImagePrivacyBoundary.discard(image) releases the transient approved image after recognition processing.

9.5 F05 — Photography Gear Inventory Management

For UC05, the presentation interface would obtain InventoryService through PhotoPilotFacade.inventory(). SD02 shows list(query) loading GearInventory through InventoryRepository.load() and using GearInventory.search(query) to return matching records. Camera, Lens, Filter, Battery, and StorageMedia represent the supported equipment categories.

For additions and edits, PhotoPilotFacade.saveGear(item) delegates to InventoryService.save(item). The service invokes Equipment.validate() and duplicates(item) before applying GearInventory.add(item) or update(item). InventoryRepository.save(inventory), implemented by JsonInventoryRepository, persists the resulting inventory. Confirmed UC04 imports enter this same deterministic inventory workflow.

Before removal, InventoryService.affectedSessions(id) uses SessionRepository.referencingGear(id) to identify historical references. After the user accepts the warning, remove(id, warningAccepted) removes the live inventory entry and saves the change. Existing GearSnapshot records preserve historical equipment information. Invalid specifications, unresolved duplicate warnings, lookup failures, and persistence errors prevent an unsupported success claim. Inventory management does not invoke Gemini and remains available when AI services fail.

9.6 F06 — AI Photography Equipment Capability Requirement Analysis

For UC06, the GUI or CLI would call PhotoPilotFacade.analyzeNeeds(id), which delegates to PlanningService.analyzeNeeds(id). SD02 shows the service retrieving the session through SessionRepository.find(id) and requiring confirmed shooting requirements before invoking RequirementService.deriveProfile(req).

PhotoPilotAgent uses Gemini to derive an EquipmentRequirementProfile containing capability needs and priorities rather than product identities. CapabilityNeed represents individual requirements. Deterministic EquipmentRequirementProfile.validate(req) checks ranges and consistency with confirmed constraints before the draft is returned for review.

Saving requires a separate explicit request through PhotoPilotFacade.planning() and PlanningService.confirmProfile(id, edited). The service reloads the session, validates the edited profile against its current requirements, and invokes the profile's confirm(). PhotographySession.attachProfile(profile) associates the confirmed value with the session, followed by SessionRepository.save(session).

Ambiguous requirements or conflicting priorities require clarification. Gemini failure, invalid edits, cancellation, or persistence failure must not be reported as a saved profile. The corrected SD02 explicitly separates analysis from confirmation and successful persistence.

9.7 F07 — Owned Gear Matching and Compatibility Validation

For UC07, PhotoPilotFacade.matchGear(id) would delegate to PlanningService.matchGear(id). SD03 retrieves the reviewed capability profile and confirmed inventory, then invokes GearMatchingService.match(profile, gear).

compatibleCandidates(gear) establishes candidate loadouts. For mandatory included UC14, CompatibilityValidator.validate(loadout) checks equipment compatibility, and CompatibilityReport.isUsable() excludes incompatible or unverified combinations. CompatibilityValidator.matchNeeds(loadout, profile) identifies unmet capability requirements.

The Strategy pattern separates ranking preferences from compatibility rules. The configured LoadoutRankingStrategy—PortabilityRanking or CoverageRanking—invokes rank(candidates, profile) and uses PhotoPilotAgent for Gemini-based comparison. GearMatchingService checks returned candidate identifiers against the validated candidates, preventing AI from introducing unverified combinations.

The result presents ranked loadouts and explicit limitations. Compatible partial matches may identify unmet needs; missing specifications require confirmation. Gemini failure prevents AI ranking without invalidating deterministic compatibility findings. SD03 returns choices for review, but the design does not expose an operation that attaches the user's selected loadout to the session. Runtime strategy-selection wiring is also left

unspecified.

9.8 F08 — Photography Technical Calculation Tool Suite

For UC08, either presentation interface would submit a CalculationInput through PhotoPilotFacade.calculate(input). SD03 shows delegation to TechnicalCalculator.calculate(input) and validation through CalculationInput.validate() before the selected calculation proceeds.

TechnicalCalculator provides depthOfField(input), hyperfocal(input), fieldOfView(input), exposureAndND(input), storage(input), and batteryCapacity(input). These operations would use deterministic Java logic and return Result<CalculationResult>. The result retains numerical values, units, assumptions, input information, and an estimate flag where applicable.

The same calculator supports agent workflows. In SD04, an approved tool invocation reaches CalculationTool.execute(argumentsJson), which delegates to TechnicalCalculator.calculate(input) and returns serialized evidence. Gemini may decide when a calculation is useful, but it does not replace the calculation itself.

Missing parameters, invalid ranges, incompatible units, and unsupported inputs produce errors rather than purported valid results. Capacity estimates must retain their assumptions. Standalone calculations require no Gemini access or independent save operation; planning may subsequently incorporate their results into a validated plan.

9.9 F09 — Location, Time, and Environment-Aware Photography Planning

For UC09, PhotoPilotFacade.assessEnvironment(id) would delegate to PlanningService.assessEnvironment(id). SD03 retrieves the session requirements and invokes EnvironmentService.assess(req).

LocationService.current(explicitConsent) accesses the optional DeviceLocationProvider only with permission. Otherwise, manual(label, lat, lon) supports user-supplied coordinates. EnvironmentService.retrieve(place, date) obtains factual weather and sunlight information through EnvironmentProvider.fetch(place, date), implemented by EnvironmentApiAdapter. EnvironmentContext.staleFields(now, maxAge) identifies outdated evidence.

PhotoPilotAgent uses Gemini to interpret the available context for the intended shoot and propose an EnvironmentAssessment. Deterministic FeasibilityValidator.checkTiming(assessment) checks timing consistency before the result is returned. ShootingWindow represents proposed shooting and arrival times.

Denied or unavailable device access permits manual input. Missing or stale environmental data must be disclosed rather than converted into precise claims. Gemini failure prevents unsupported interpretation while factual retrieval remains independent. SD03 returns guidance for subsequent planning; it does not show a separate operation persisting the assessment.

9.10 F10 — AI Shoot Plan Generation and Interactive Modification

For UC10 and UC11, the GUI or CLI would use PhotoPilotFacade.generatePlan(id) or proposeChange(id, text), delegating to PlanningService.generate(id) or proposeChange(id, text). SD04 retrieves session requirements, the reviewed capability profile, inventory, and matched loadouts, assembling available environmental and calculation evidence. HistoryRetriever.contextFor(id) supplies relevant history to the agent.

PhotoPilotAgent.run(task, context, type) coordinates Gemini reasoning and bounded tool

use. ToolRegistry.execute(call) checks permitted tool calls, and CalculationTool supplies deterministic evidence. Schema checking precedes PlanValidator.validate(plan, req), including UC14 compatibility checks and numerical validation. A valid proposal remains a preview until approval.

For revision, SD04 illustrates a ReplaceLoadoutCommand. PlanEditor.preview(base, change) creates a copy through ShootPlan.copyForRevision(), then invokes PlanCommand.applyTo(draft). PlanValidator.validateChange(base, next) checks compatibility, calculations, fixed constraints, and unaffected decisions.

Explicit approval calls PlanningService.approve(id, plan) through the Facade's planning accessor. It reloads the session, rejects stale or foreign revisions, checks editing permission, and repeats required validation. appendApprovedPlan(plan) stages the approved version; initial approval invokes apply(PLAN_APPROVED) under the State pattern. The repository saves the plan and state together. Rejection, ambiguity, API failure, validation failure, or failed persistence preserves the stored base. Approval does not reapply the command.

9.11 F11 — AI Post-Shoot Analysis and Review

For UC12, PhotoPilotFacade.analyze(id, files) would delegate to ReviewService.analyze(id, files). SD05 loads the completed session, then uses MetadataExtractor, implemented by ExifLibraryAdapter, to check supports(file) and extract(files). ExifRecord.validate() checks extracted evidence, while StatisticsCalculator.summarize(records) deterministically produces ExifStatistics. Missing measurements are excluded from affected calculations. Successful summaries are attached through PhotographySession.attachStatistics(stats) and persisted through SessionRepository.save(session).

UC13 is a separate request through PhotoPilotFacade.review(id). ReviewService.review(id) reloads saved evidence, uses StatisticsCalculator.compare(stats, plan), and invokes PhotoPilotAgent with available history for Gemini-based interpretation. The review must acknowledge evidence limitations and cannot infer aesthetic quality.

Explicit saving calls ReviewService.save(id, review) through PhotoPilotFacade.reviews(). appendReview(review) and apply(REVIEW_SAVED) stage the review and State transition before saving the aggregate.

Unreadable files do not invalidate remaining usable records. No-data results must not overwrite saved statistics. Gemini failure leaves deterministic analysis available; persistence failure must not be reported as a saved review or completed transition.